Travel Planner Agent

A Teaching Book For The Full Project

Detailed architecture, concepts, runtime flow, folder-by-folder explanation, and file-by-file explanation of the repository as it exists today.

Generated on 2026-07-04 11:34 from the repository root at `/Users/naveen.kumar.p/Desktop/pprojects/travel-planner-agent`.

The layered architecture image from the repository docs. The book repeatedly maps concepts back to this stack view.

Scope note: the directory and file encyclopedia explain project-owned source, docs, tests, scripts, and config. Generated folders, caches, build outputs, and external dependencies are intentionally excluded.

Table Of Contents

- 1. Why This Project Exists
- 2. How To Read This Repository
- 3. Product Flow At A Glance
- 4. Technology Stack And Why Each Piece Matters
- 5. Top-Level Repository Layout
- 6. Local Startup And Environment Strategy
- 7. Backend Layering Philosophy
- 8. FastAPI Entry And Route Surface
- 9. Configuration, Security, And Request Context
- 10. Domain And Application Contracts
- 11. Core Planner Schemas
- 12. Clarification Copilot Design
- 13. The Original Sequential Planner Graph
- 14. Specialist Nodes And Artifact Production
- 15. From Sequential Pipeline To Coordinator Runtime
- 16. Coordination Ledger, Roles, Tasks, And Topology
- 17. Tooling And External Providers
- 18. Persistence And Database Evolution
- 19. Async Runtime, Worker, And Redis
- 20. Audit, Observability, And Operator Review
- 21. Frontend Architecture
- 22. Authentication And User Profile Direction
- 23. Testing Strategy
- 24. CI, Deployment, And Operational Docs
- 25. What This Project Teaches About Agentic Engineering
- 26. Commercialization Gap Analysis
- 27. Study Plan For Learners
- 28. Extension Exercises
- 29. Directory Encyclopedia
- 30. File Encyclopedia
- 31. End-To-End Request Lifecycle Walkthrough

- 32. Workflow Runtime Semantics
- 33. Frontend Screen-By-Screen Tour
- 34. Database Table And Migration Story
- 35. Test-Driven Reading Guide
- 36. Recommended Refactoring Principles For This Repo
- 37. Guided Study Questions
- 38. Guided Code-Reading Prompts
- 39. Glossary Of Project Terms
- 40. Final Reading Checklist
- 41. Commercial Hardening Checklist
- 42. Module-by-Module Teaching Notes
- 43. Design Principles To Carry Forward

Repository Snapshot

Project-owned files included	218
Project-owned directories included	67
Backend runtime style	FastAPI + LangGraph + custom coordinator + RQ worker
Frontend runtime style	React + TypeScript
Primary storage	Postgres
Async coordination substrate	Redis

Sample Tree

```
travel-planner-agent/
■■■ .github
■   ■■■ workflows
■       ■■■ ci.yml
■       ■■■ deploy-production.yml
■       ■■■ deploy-staging.yml
■       ■■■ secrets-scan.yml
■■■ backend
■   ■■■ alembic
■   ■   ■■■ versions
■   ■   ■■■ env.py
■   ■   ■■■ script.py.mako
■   ■■■ evals
■   ■   ■■■ regression_baseline.json
■   ■■■ scripts
■   ■   ■■■ load_test.py
■   ■■■ src
■   ■   ■■■ agents
■   ■   ■■■ api
■   ■   ■■■ application
■   ■   ■■■ core
■   ■   ■■■ db
■   ■   ■■■ domain
■   ■   ■■■ evals
```

```
│   │   ├── messaging
│   │   ├── models
│   │   ├── persistence
│   │   ├── providers
│   │   ├── repositories
│   │   ├── services
│   │   ├── utils
│   │   ├── workers
│   │   ├── __init__.py
│   │   ├── bootstrap.py
│   │   └── main.py
│   ├── tests
│   │   ├── conftest.py
│   │   ├── test_api_security.py
│   │   ├── test_application_layers.py
│   │   ├── test_config.py
│   │   ├── test_destination_research_agent.py
│   │   ├── test_domain_layers.py
│   │   ├── test_eval_suite.py
│   │   ├── test_multi_agent_coordination.py
│   │   ├── test_observability.py
│   │   ├── test_operator_review.py
│   │   ├── test_provider_clients.py
│   │   ├── test_provider_live_smoke.py
│   │   ├── test_reference_links.py
│   │   ├── test_runtime_resilience.py
│   │   ├── test_tooling_layer.py
│   │   └── test_workflow_runtime.py
│   ├── .dockerignore
│   ├── .env
│   ├── .env.dev
│   ├── .env.dev.example
│   ├── .env.prod.example
│   ├── .env.staging.example
│   ├── alembic.ini
│   ├── Dockerfile
│   ├── pyproject.toml
│   ├── requirements-dev.txt
│   └── requirements.txt
├── docs
│   ├── assets
│   │   └── layered-architecture.png
│   ├── operations
│   │   ├── deployment.md
│   │   ├── load-and-resilience.md
│   │   ├── runbooks.md
│   │   └── slo.md
│   ├── agent-communication.md
│   ├── agent-flow.md
│   ├── api-contract.md
│   ├── architecture.md
│   ├── layered-architecture.md
│   └── true-multi-agent-architecture.md
├── frontend
│   ├── public
│   │   └── index.html
│   ├── src
│   │   ├── app
│   │   ├── assets
│   │   ├── components
│   │   ├── features
│   │   ├── hooks
│   │   ├── layouts
│   │   ├── lib
│   │   ├── services
│   │   ├── types
│   │   ├── App.tsx
│   │   ├── index.css
│   │   └── index.tsx
```

```
■   ■   ■■■ react-app-env.d.ts
■   ■■■ .dockerignore
■   ■■■ .env.local
■   ■■■ Dockerfile
■   ■■■ package-lock.json
■   ■■■ package.json
■   ■■■ tsconfig.json
■■■ scripts
■   ■■■ generate_teaching_book_pdf.py
■   ■■■ start-local-stack.sh
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ .nvmrc
■■■ .pre-commit-config.yaml
■■■ .python-version
■■■ docker-compose.yml
■■■ package-lock.json
■■■ README.md
■■■ TRAVEL_PLANNER_TEACHING_BOOK.pdf
```

Chapter 1. Why This Project Exists

Travel Planner Agent is best understood as two projects welded together. On the product side, it is a traveler-facing planning experience that takes a destination, budget, dates, interests, and travel constraints and tries to produce a coherent plan. On the engineering side, it is an agentic-systems reference implementation that explores layered architecture, workflow orchestration, coordinator-led multi-agent execution, async job handling, persistence, observability, governance, and frontend presentation.

Many repositories show only a prompt and a response. This repository is more valuable because it exposes the machinery in between. It includes explicit request schemas, route handlers, validation, planner state, workflow runtime records, job queues, agent coordination contracts, evidence tracking, review outputs, and a browser client that renders the artifacts across multiple screens. That makes it suitable for teaching. A learner can read it as a product, as a backend service, as a workflow engine, or as a case study in how a prototype moves toward a commercial-grade system.

The central teaching idea of this book is that good agentic systems are not only about model prompts. They are about disciplined interfaces between layers. The trip request needs structure. The graph needs typed state. Agents need bounded contracts. The queue needs idempotent behavior. The frontend needs a stable API client. Audit and observability need explicit storage. Once those layers are in place, the model becomes one component in a wider system instead of the entire architecture.

This book therefore teaches the project in two parallel dimensions. First, it explains what each part does in practical terms. Second, it explains why the part exists from a systems-design perspective. Every chapter tries to answer both questions because understanding only the surface behavior is not enough to extend the repository safely.

Chapter 2. How To Read This Repository

Start from the outside and move inward. The outer layer is the user interface and the HTTP API. Those reveal the promises the system makes to users and to callers. The middle layer is orchestration: the planner graph, the coordinator runtime, and the clarification copilot. The inner layer is contracts and infrastructure: schemas, persistence, providers, and tests. If you reverse that order and dive into isolated utility modules first, the repository feels larger and more fragmented than it actually is.

Keep a mental distinction between current behavior and target architecture. The repo still contains the original sequential LangGraph pipeline in `backend/src/agents/travel_planner/graph.py`, but it also contains the newer custom coordinator runtime in `backend/src/agents/travel_planner/multi_agent/runtime.py`. This is not duplication by accident. It is a design history made visible. The sequential graph teaches the bounded-workflow model; the coordinator runtime teaches the shift toward true multi-agent behavior and selective parallelism.

Read the repository with three questions in mind. What is the boundary of this module. What state does it own or transform. What are the failure modes if it is wrong. Those questions are more useful than simply asking what the file does. For example, a route file may look small, but if it shapes security or response contracts incorrectly it can destabilize the whole frontend. A schema file may look boring, but if it relaxes validation too far the planner runtime becomes unpredictable. Teaching value often hides in the quiet files.

This book also includes a directory and file encyclopedia so no project-owned artifact is left unexplained. Generated, vendor, or cache directories such as `frontend/node_modules`, `frontend/build`, `.git`, `.ruff_cache`, and `.pytest_cache` are intentionally excluded from the appendix because they are external or generated rather than authored system design.

Chapter 3. Product Flow At A Glance

The user journey begins on the New Trip page. The traveler enters origin, destination, travel dates, budget, traveler count, trip purpose, pace, and preference-oriented notes. The UI performs local readiness checks so obviously broken input does not proceed.

The next major stop is the Clarification page. Instead of immediately running the heavy workflow, the system now uses a clarification copilot to ask focused follow-up questions. The copilot uses the base trip request, destination-aware signals, and accumulated answers to refine a `clarification_profile`. This is a crucial design improvement because it separates raw intake from high-signal disambiguation.

After clarification, the async workflow is launched. The backend creates a job and a run record, the worker begins execution, and the frontend polls workflow runtime endpoints. Research, itinerary planning, parallel specialists, budget review, review, and governance become visible stages rather than hidden backend behavior. That visibility matters because agentic applications feel more trustworthy when progress is inspectable.

Finally, the user navigates through research, itinerary, budget, and review screens. These are not arbitrary tabs. They correspond to major artifact classes produced by the backend. The frontend is therefore not inventing its own domain; it is projecting backend contracts into human-readable views. Good full-stack design often looks like this: one shared conceptual model rendered differently at different layers.

Chapter 4. Technology Stack And Why Each Piece Matters

The frontend is built with React and TypeScript. React is responsible for page composition, user interactions, and client-side polling of runtime state. TypeScript makes the integration contract visible by encoding request and response shapes in the frontend. That reduces the chance that backend contract changes silently break the UI.

The backend is built with FastAPI and Pydantic. FastAPI gives a clean request-response boundary, dependency injection, and automatic JSON contract handling. Pydantic provides strict typed models for trip requests, clarification answers, runtime responses, audit payloads, and other cross-layer data. This stack is a pragmatic choice for an agentic app because it allows rapid development without sacrificing model validation.

LangGraph is used in two ways. First, the repository preserves the original sequential planner graph that demonstrates state-based workflow orchestration. Second, the newer coordinator runtime uses LangGraph as the control framework around a custom coordination ledger. The important lesson is that an orchestration library does not decide your architecture for you; it gives you primitives. The repo shows both a linear pipeline and a coordinator-led runtime built on those primitives.

Postgres stores durable state. Redis supports queueing and async job processing. RQ workers execute planner runs outside the web request path. Tavily, OpenAI, WeatherAPI, SerpApi, and Aviationstack sit behind provider abstractions. In combination, the stack teaches a common production pattern: UI plus API plus workflow plus queue plus persistence plus external providers. Each piece is familiar; the educational value comes from how the repository integrates them coherently.

Chapter 5. Top-Level Repository Layout

The repository root contains the cross-cutting artifacts. `README.md` is the human landing page. `docker-compose.yml` is the local runtime topology. `.env.example`, `.nvmrc`, and `.python-version` help standardize developer setup. `.github/workflows` defines CI and deployment behavior. The root `scripts` directory contains helper scripts that operate on the whole stack rather than on one application boundary.

The `backend` folder is the Python service boundary. It contains its own dependency manifests, Dockerfile, migration history, environment samples, tests, and the complete source tree. This is where orchestration, persistence, provider integration, services, and business logic live.

The `frontend` folder is the React application boundary. It has its own package manifest, TypeScript config, Dockerfile, and source tree. UI routes, reusable components, shared layout, and the client API layer live here.

The `docs` folder captures architectural and operational knowledge outside the code. That is a sign of maturity. Code explains implementation; docs explain system intent, operational standards, and architectural evolution. Strong engineering organizations need both.

Chapter 6. Local Startup And Environment Strategy

The simplest way to run this project locally is through Docker Compose. The compose file starts Postgres, Redis, the backend API, the background worker, and the frontend. This is the shortest path to a realistic environment because the backend genuinely depends on both durable storage and a queueing substrate.

The backend reads layered environment files, including `backend/.env`, plus environment-specific overlays such as `backend/.env.dev`. This lets the same code run in different modes without inlining secrets or environment branching directly into business logic. The frontend uses `REACT_APP_API_BASE_URL` to know which backend to call, keeping the browser client decoupled from hardcoded local URLs.

The startup sequence matters. Postgres and Redis must become healthy before the backend and worker start. The backend applies Alembic migrations before serving traffic. The worker connects to Redis and listens on the planner queue. The frontend expects the backend to exist. The repository makes that order visible both in `docker-compose.yml` and in the helper startup script.

Teaching takeaway: orchestration bugs often come from ignoring startup dependency order. This repository avoids that by expressing dependency readiness at the container level and by keeping a dedicated readiness service on the backend side as an extra health boundary.

Key Local Commands

```
docker compose up --build

cd backend && python3 -m alembic upgrade head && python3 -m uvicorn src.main:app --reload --port 8

cd frontend && npm start

bash scripts/start-local-stack.sh
```

Chapter 7. Backend Layering Philosophy

The backend follows a layered structure that is intentionally more disciplined than a quick prototype. The API layer handles HTTP. The application layer defines response schemas and use cases. The domain layer holds stable business concepts. Persistence repositories isolate storage. Providers isolate external API clients. Services coordinate behavior. Agent modules own planning runtime logic. This separation does not make the system academic; it makes change safer.

Consider what happens when a new frontend page needs a richer run trace. Without layering, route code might query the database directly, shape JSON ad hoc, and mix transport decisions with persistence details. In this repository, a better path exists: the API route calls a service, the service uses repositories, application mappers shape response models, and the frontend consumes typed output. That may feel like more files, but the long-term benefit is local reasoning.

Layering is especially important in agentic systems because prompts, provider calls, queueing, persistence, and user-facing rendering all evolve at different speeds. A repo that puts them into one giant service file becomes impossible to extend safely. This project's structure is therefore itself part of the teaching material. It demonstrates that agentic applications still benefit from ordinary software-engineering discipline.

The main caveat is that layered systems require naming discipline. Not every helper deserves a new layer. The repo is strongest where file roles are explicit and where transport, orchestration, and domain concepts remain distinct. When extending the system, readers should preserve that spirit rather than blindly creating more modules.

Chapter 8. FastAPI Entry And Route Surface

The backend process starts in `backend/src/main.py`, which creates the FastAPI app, installs middleware, and attaches the API router. This file should stay lean. Its job is process assembly, not planner behavior.

The main route definitions live in `backend/src/api/main.py`. This file is one of the best learning surfaces in the backend because it reveals nearly all system capabilities: health endpoints, auth registration and login, trip creation, async trip creation, clarification copilot, trip fetch, job fetch, admin review endpoints, observability endpoints, run trace endpoints, and workflow step endpoints.

Each route follows the same discipline. It accepts a typed request model, resolves dependencies from the container, enforces role-based access where required, invokes a use case or service, and returns a typed response model. This keeps HTTP concerns explicit and prevents transport-level logic from leaking all over the backend.

A crucial design choice is the separation between synchronous and asynchronous trip creation. `POST /api/trips` can produce a direct trip response. `POST /api/trips/async` creates a job and run that will be executed in the background. For teaching purposes, that split illustrates how the same business domain can support different latency and UX models without inventing a new domain structure.

Main Route Groups

- Health and readiness endpoints prove service and dependency status.
- Auth endpoints create and retrieve user identity context.
- Trip endpoints support sync and async planning flows.
- Clarification endpoint powers the guided copilot before workflow launch.
- Admin, trace, and observability endpoints support operator and audit use cases.

Chapter 9. Configuration, Security, And Request Context

Backend infrastructure modules in `backend/src/core` are not glamorous, but they control the safety and predictability of the whole system. `config.py` resolves structured settings. `logging.py` standardizes log creation. `request_context.py` manages per-request metadata such as request ids. `response_shaping.py` normalizes outgoing responses. `security.py` handles actor context, roles, and rate limiting. `redaction.py` exists so sensitive data can be treated explicitly rather than by wishful thinking.

The configuration pattern is especially useful pedagogically. Instead of peppering `os.getenv` across the codebase, the repo centralizes settings. That means provider credentials, Redis URLs, database URLs, request timeouts, and security toggles become a coherent model. When students later add new providers or deployment environments, they have an obvious home for those changes.

Security is also worth studying because this repo is not only a toy planner. It contains admin routes, user-level routes, and operator review capabilities. The `require_roles` pattern makes authorization readable at the route layer, while actor metadata can still be consumed deeper down for audit events or approval records.

Good request-context handling matters for observability and auditability. If a request id is attached early and preserved, downstream logs and audit entries can be correlated. In agentic applications, that is the difference between a debuggable run and an opaque failure.

Chapter 10. Domain And Application Contracts

The domain layer defines stable business nouns. Trips, jobs, workflows, workflow steps, and audit records are represented in `backend/src/domain`. These modules should not care whether the caller is FastAPI, a worker process, or a unit test. They exist to model business reality in a reusable way.

The application layer sits one step outward. It contains schemas and mappers for how the outside world should consume those domain concepts. For example, workflow run repositories may store one shape internally, but `backend/src/application/workflows/schemas.py` defines what the API should promise to clients. This prevents direct leakage of persistence representation into transport contracts.

One of the strongest educational moves in this repo is the use of separate mapper modules. Beginners often think mappers are unnecessary indirection. In practice, they become essential as soon as API payloads, internal models, and database records stop being identical. The mapper makes that transformation explicit instead of spreading it across routes and services.

Domain and application layering also future-proofs the codebase. If one day this planner exposes a second API, a mobile backend, or a partner integration surface, it can reuse the domain while shaping a different application contract. That is exactly the sort of flexibility commercial systems need.

Chapter 11. Core Planner Schemas

`backend/src/agents/travel_planner/schemas.py` is one of the most important files in the repository. It defines the central trip request contract, clarification structures, planner output artifacts, confidence-carrying assessments, and cross-layer models consumed by both the backend and the frontend.

Study `TripRequest` carefully. It includes origin, destination, dates, traveler count, trip purpose, total budget, budget tier, pace, interests, accommodation preference, transport preference, traveler constraints, and a structured `clarification_profile`. This is not just data collection. It is the information architecture of the planner. If the request is weak or underspecified, every downstream agent becomes guesswork.

The validation behavior matters as much as the field list. Dates must be ordered correctly. Trip duration is capped. Text fields are normalized. Interests are deduplicated. Constraint notes are trimmed. These details protect the planner from messy input and reduce accidental prompt variability.

The same file also shows how the system treats clarifications as first-class structured data rather than loose chat history. That is a strong design choice. Free-form chat can be human-friendly, but structured clarification answers are far easier for downstream specialists to consume reliably.

Chapter 12. Clarification Copilot Design

`backend/src/services/clarification_copilot_service.py` demonstrates an elegant pattern for low-latency pre-planning intelligence. Instead of launching the full planning workflow immediately, the service first enriches the base request with a guided question sequence. It uses Tavily-derived destination signals when available, falls back gracefully when not, applies answers to a typed clarification profile, and decides the next best question.

This service uses LangGraph in a very lightweight way. The graph only has two stages: collect destination signals, then build the next question. That shows an important architectural truth: not every graph needs to be large. LangGraph is useful even for small state transitions when the state model is explicit and the flow may later evolve.

The copilot embodies a product insight as well as a technical one. Clarification is valuable because many travel inputs are underdetermined. A destination alone cannot tell the planner whether the traveler wants a walkable base, a scenic retreat, a celebration-first memory, or late-night flexibility. By asking a few specific questions, the system turns a generic form into a richer planning contract.

This chapter is also a warning against premature autonomy. A commercial system should not treat every free-form user input as enough to trigger expensive downstream agents. A clarification layer can improve quality, reduce waste, and make later outputs feel more personal without requiring a heavier backend runtime.

Chapter 13. The Original Sequential Planner Graph

`backend/src/agents/travel_planner/graph.py` preserves the original sequential planner runtime. It builds a `TravelPlannerGraph`, registers node handlers, defines the entry step, routes after clarification, executes nodes in order, records audit events, and stores run summaries and governance flags.

This file is pedagogically useful because it is simple enough to understand in one sitting. Clarification comes first. If clarification blocks progress, the graph ends early. Otherwise the workflow proceeds through research, itinerary, stay recommendation, local transport, food recommendation, budget optimization, solo-women safety, review, and governance. The order is deterministic and easy to trace.

The graph also illustrates a classic state-machine principle. Nodes do not talk directly to each other. Instead they read and update shared state. That keeps node contracts cleaner and makes graph-level routing decisions easier to reason about. The downside is that true autonomy is limited because every specialist behaves like a step in a pipeline, not an agent capable of selective delegation.

If you are teaching workflow fundamentals, start here. If you are teaching autonomy and latency optimization, move on to the coordinator runtime. Both views matter because many real systems begin with the first and only later earn the complexity of the second.

Chapter 14. Specialist Nodes And Artifact Production

`backend/src/agents/travel_planner/nodes.py` is where the planner becomes concrete. This large module contains the specialist logic for clarification validation, research signal gathering, destination research, itinerary planning, stay recommendation, local transport, food recommendation, budget optimization, safety advice, review, and governance. In other words, this file is the artifact factory of the backend.

The design strength of the node layer is bounded responsibility. Each node owns one artifact class or one routing concern. The itinerary node does not also own approval logic. The budget node does not own page rendering. The review node evaluates coherence and issues. That boundedness is why the same node logic can later be reused under the coordinator runtime through adapters.

Another valuable lesson here is output typing. Each specialist is expected to produce a model-shaped result with confidence and assumptions rather than a blob of text. That makes later aggregation, UI rendering, and governance much easier. It is far simpler to compute readable budget cards or itinerary timelines when the backend already thinks in structured artifacts.

Students reading this file should pay attention to three dimensions at once: how prompts are constructed, how provider fallbacks are handled, and how evidence or confidence is attached to output. Those three dimensions together decide whether an LLM feature feels like a trustworthy subsystem or a fragile demo.

Chapter 15. From Sequential Pipeline To Coordinator Runtime

The biggest architectural evolution in the repo is the move from a fixed chain to a custom coordinator-led runtime. This lives in `backend/src/agents/travel_planner/multi_agent`. The idea is not to let agents chatter arbitrarily. The idea is to let a coordinator choose which bounded specialist should work next, when some specialists can run in parallel, and when rework or governance checks are required.

`runtime.py` is the heart of that system. It defines a `CoordinatorRuntime` with a LangGraph wrapper, a decision node, specialist nodes, and a `parallel_batch` execution path. It keeps a typed `CoordinationLedger` alongside the planner context and emits progress updates so the frontend can display meaningful workflow motion.

The coordinator runtime improves both architecture and latency. After itinerary generation, several specialists such as stay, transport, food, and safety can be released together. That shortens wall-clock time without discarding dependency order. Budget, review, and governance remain fan-in stages because they depend on combined outputs. This is a practical example of selective parallelism rather than naive concurrency.

The teaching message is important: autonomy should be constrained by contracts and topology. The repo does not worship agents as magical independent minds. It treats them as specialized workers under a coordinator and a ledger. That is a much more defensible design for real systems.

Parallelism Model

```
Clarification -> Research -> Itinerary -> [Stay | Transport | Food | Safety] -> Budget -> Review -
```

Chapter 16. Coordination Ledger, Roles, Tasks, And Topology

The coordinator runtime works because its state is richer than the older shared planner context. `backend/src/agents/travel_planner/multi_agent/schemas.py` defines roles, tasks, messages, artifacts, and ledger metadata. `topology.py` defines allowed delegation and initial task seeding. `coordinator.py` converts ledger state into routing decisions.

This trio of files is where the repository starts to resemble a true multi-agent system rather than a renamed pipeline. The ledger captures objective, task board, message log, artifact store, and iteration boundaries. Roles such as Coordinator, Clarification, Destination Research, Itinerary, Stay, Transport, Food, Budget, Safety, Review, Governance, and Human Operator become explicit rather than implied.

Topology matters because it prevents uncontrolled recursion. For example, not every specialist is allowed to delegate to every other specialist. The coordinator has wide authority; a food specialist does not suddenly get to trigger governance. This explicit delegation matrix is one of the most commercially useful ideas in the repo because it turns autonomy into policy rather than improvisation.

Students should compare this design to the original graph. The graph is simpler. The ledger is richer. The coordinator runtime earns its complexity by making parallel work, controlled revision, and explicit runtime traces possible.

Chapter 17. Tooling And External Providers

The backend has a dedicated provider layer in `backend/src/providers` and a planner-specific tooling layer in `backend/src/agents/travel_planner/tooling`. This distinction is subtle but valuable. Provider modules know how to talk to third-party APIs. Tooling modules know how those provider capabilities are exposed to planner specialists in a policy-aware way.

The provider factory currently supports Tavily, WeatherAPI, OpenAI, SerpApi, and Aviationstack. The factory pattern keeps provider instantiation centralized, making it easier to manage secrets, timeouts, and future provider additions. A planner node should not need to know how to construct all its clients from raw environment variables.

The tooling layer adds another level of maintainability. It gives the planner registry, base contracts, travel-specific helper tools, search tools, provider-binding helpers, and validation contracts. In a commercial codebase this is where rate limits, allowlists, and safety boundaries often become explicit.

A central lesson here is that external APIs should never leak straight into prompts or route handlers. They should be wrapped, normalized, and governed. The repository does not do this perfectly in every spot, but its direction is correct and instructive.

Chapter 18. Persistence And Database Evolution

Durable application state lives in Postgres, and database evolution is tracked through Alembic migrations. The repository has models, repositories, and migration history for trips, jobs, workflow runs, workflow steps, audit events, approval and evidence fields, and user records. That makes the planner inspectable over time instead of functioning as a stateless prompt gateway.

`backend/src/db/models.py` defines the relational models. Repository implementations in `backend/src/persistence/postgres` expose storage behavior for each aggregate. Memory repositories in `backend/src/persistence/memory` provide lighter alternatives for tests or simplified execution. The separation between storage contract and implementation is essential because it lets services and runtimes depend on abstractions rather than one database directly.

Migration history deserves real attention. The version files show how the repo matured: base trip and job tables came first, then trip state and audit events, then workflow step tracking, then approval and evidence fields, then workflow runs and runtime fields, and finally users. This sequence tells the story of a system moving from prototype response generation toward operational and product maturity.

Many agentic demos skip database evolution entirely. This repository does not, and that is a strength. Persistent state is not overhead; it is what allows review queues, reruns, traces, analytics, and user-facing credibility features.

Chapter 19. Async Runtime, Worker, And Redis

The asynchronous path is one of the repository's most practically important features. The backend does not force all planning work into the web request lifecycle. Instead it can enqueue jobs through Redis-backed RQ and execute them in a worker process. This is the right choice for multi-step planning workloads that may involve provider latency, retries, or parallel specialist execution.

`backend/src/messaging/redis_queue.py` wires the queue. `background_jobs.py` defines background execution concerns. `backend/src/workers/workflow_worker.py` is the worker-side entrypoint. `backend/src/services/workflow_runtime_service.py` is the key orchestrator that creates runs, step records, queue jobs, and state transitions.

Redis is not used here as a generic fashionable component. It exists because the system needs a lightweight queue backend and shared async coordination infrastructure. The project could theoretically use another queueing system later, but the teaching pattern would remain: keep heavy or retry-prone workflow execution away from the synchronous API path.

The frontend benefits from this design because it can show progress rather than freezing on one request. The backend benefits because retries, dead lettering, and monitoring become explicit runtime concerns. This is exactly the sort of architecture step that moves a demo toward production thinking.

Chapter 20. Audit, Observability, And Operator Review

Trustworthy agentic systems need more than final answers. They need traces, audit records, review surfaces, and operational metrics. This repository implements those concerns through services such as `audit_service.py`, `observability_service.py`, `operator_review_service.py`, and supporting repositories plus application schemas.

Audit records capture events such as request receipt, response return, and node-level activity. Observability surfaces run traces, alerts, and metrics. Operator review introduces the idea that some outputs may need human approval or at least a dedicated queue for attention. This is not just enterprise ornamentation. It is how systems become governable when LLM output quality varies.

The review screen in the frontend and the admin routes in the backend are where this architecture becomes visible to end users and operators. If a run is low confidence, exceeds budget, or uses fallback behavior, that should not stay buried in logs. The repository therefore teaches an important product lesson: decision quality signals must become user-visible artifacts.

Readers who want to commercialize the project should treat this layer as a starting point rather than a finished story. The main thing to preserve is the principle: every important automated decision path should leave an explainable trail.

Chapter 21. Frontend Architecture

The frontend is a route-oriented React application. `App.tsx` defines navigation, `index.tsx` mounts the application, `AppLayout.tsx` provides shared structure, and route components under `frontend/src/app` implement the major planner screens. Reusable pieces such as cards, sidebars, export panels, chat widgets, and status badges live under `frontend/src/components`.

The central integration module is `frontend/src/lib/planner.ts`. It contains typed request and response models, fetch helpers, localStorage helpers, run-control helpers, and client-side state persistence. This is the file that translates backend contracts into a frontend-usable language. Without it, each page would reimplement API calls and local state behavior inconsistently.

One of the most interesting UI flows is the shift of runtime visibility toward the Clarification page. The repo now leans toward a product story where the user first fills a generic form, then refines intent through clarification, then watches workflow execution. That is a cleaner mental model than immediately launching an opaque backend run from a raw form.

The frontend teaches a broader lesson as well. Even in an agentic app, the UI should not be a thin text dump. It should organize outputs into domain-shaped screens such as research, itinerary, budget, and review. That helps users build trust and lets the engineering system stay modular underneath.

Chapter 22. Authentication And User Profile Direction

The repository includes user registration and login endpoints plus frontend login and signup screens. This is a significant step beyond a pure anonymous demo because it introduces identity, actor roles, and the beginning of user-specific persistence.

`backend/src/services/auth_service.py`, `backend/src/application/auth/schemas.py`, `backend/src/persistence/postgres/user_repository.py`, and the users-table migration together implement the backend side. On the frontend, the login-related screens are under `frontend/src/app/login`. This feature set is still lightweight compared with a full commercial identity platform, but it establishes the architectural seams needed for one.

The educational value is twofold. First, it shows how quickly an agentic app stops being only about prompts once user identity enters the picture. Second, it demonstrates how auth has to cross layers cleanly: database model, repository, service, API route, frontend form, and role-based backend protection.

For a commercial future, this area would likely expand into stronger password policy, session management, admin controls, profile editing, email verification, and tenant boundaries. The current code does not complete that journey, but it shows where the journey should continue.

Chapter 23. Testing Strategy

The backend test suite is broad enough to be educational in its own right. There are tests for domain layers, application layers, configuration, providers, API security, observability, workflow runtime, runtime resilience, destination research, reference links, multi-agent coordination, tooling, and operator review. This coverage pattern teaches students what kinds of surfaces matter in an agentic codebase.

Notice that tests are not only checking whether an endpoint returns 200. They also verify orchestration behavior, delegation rules, artifact population, and resilience behavior. That is exactly where agentic apps tend to fail if they are tested too superficially.

The frontend currently has a smaller testing surface focused on the planner library. That is a realistic imbalance in many early-stage projects. The higher-risk logic often lives in the backend orchestration layer, so that is where most of the automated confidence was initially invested.

A strong study exercise is to map every major backend folder to at least one related test file. Doing so reveals whether important runtime behavior is protected. It also shows where future tests would add the most value, especially around richer UI interactions and approval workflows.

Suggested Test Reading Order

- Read `test_config.py` to understand settings expectations.
- Read `test_workflow_runtime.py` and `test_multi_agent_coordination.py` to understand orchestration guarantees.
- Read `test_observability.py` and `test_operator_review.py` to understand operational surfaces.
- Read provider and research tests to understand external dependency boundaries.

Chapter 24. CI, Deployment, And Operational Docs

The `.github/workflows` directory contains the repository's automated delivery narratives. CI validates quality. Deployment workflows express environment promotion. Secret scanning adds governance. Together they show that the codebase is thinking beyond local execution.

The `docs/operations` directory is equally instructive. SLOs define expectations. Deployment docs explain release posture. Runbooks describe recovery motions. Load and resilience documentation clarifies how the system should behave under stress. These documents are not secondary to the code; they complete the system story.

Students often underestimate operational documentation because it does not compile. In reality, operational clarity is what allows a team to handle incidents, onboard contributors, and evolve a service without tribal knowledge dominating everything. This repository is stronger because it gives operations a home.

Commercial readiness is therefore not only a matter of adding more agents or providers. It is also a matter of repeatable pipelines, documented runbooks, and visible SLO-oriented thinking. Those are the habits the repo is beginning to encode.

Chapter 25. What This Project Teaches About Agentic Engineering

First, agentic systems are software systems. The flashy part is model output, but the durable part is contracts, state, queues, persistence, and user interfaces. This project makes that visible by giving equal importance to schemas, repositories, services, and runtime records.

Second, autonomy should be earned. The repository does not pretend that every specialist should run freely. It introduces a coordinator, a topology, a ledger, and bounded delegation. That is the right instinct. When systems become more autonomous, explicit policy becomes more important, not less.

Third, user trust comes from visibility. Clarification flows, progress panels, review summaries, governance flags, evidence, and audit traces are not optional polish. They are how users decide whether the system is credible.

Fourth, incremental evolution is healthy. The coexistence of the old sequential graph and the newer coordinator runtime is not architectural impurity; it is migration made visible. Good engineering often means carrying the old model long enough to learn from it while building the new one beside it.

Chapter 26. Commercialization Gap Analysis

This repository is meaningfully beyond a toy demo, but it is not yet a finished commercial product. It has clear strengths: typed contracts, async runs, workflow records, coordinator-led specialist execution, auth scaffolding, docs, tests, and a multi-screen UI. Those are serious foundations.

The main gaps are depth and hardening. Identity and sessions need to mature. Audit trails need broader exposure and stronger storage policies. Approval and governance likely need richer operator tooling. Provider usage should be rate-limited, cached, and monitored more aggressively. Error surfaces should be even more user-friendly. Observability should become dashboard-grade rather than primarily service-grade. The frontend needs more test coverage and richer failure handling.

Product-wise, commercialization would also require stronger booking-link hygiene, better source credibility handling, policy-aware recommendations, and a clear monetization strategy. Technically, deployment, secret rotation, usage analytics, and tenant boundaries would need more depth.

The key teaching conclusion is optimistic: the repo is not empty infrastructure pretending to be product. It already contains the right kinds of seams. Commercialization is now a matter of finishing layers, tightening contracts, and operationalizing decisions rather than inventing an architecture from scratch.

Chapter 27. Study Plan For Learners

Week one should focus on the outside layers. Read `README.md`, `docs/architecture.md`, `docker-compose.yml`, `backend/src/api/main.py`, and `frontend/src/lib/planner.ts`. Run the stack and correlate API endpoints with frontend behavior.

Week two should focus on planner contracts and runtime mechanics. Read `backend/src/agents/travel_planner/schemas.py`, `state.py`, `graph.py`, `nodes.py`, and `services/workflow_runtime_service.py`. Try to trace a trip from request submission to persisted run record to frontend rendering.

Week three should focus on the coordinator runtime. Read everything under `backend/src/agents/travel_planner/multi_agent` and the related tests. Compare it mentally with the original sequential graph. Ask where parallelism is allowed and why.

Week four should focus on persistence, observability, auth, and docs. Read migrations, repositories, auth service, observability service, operator review service, and operations docs. At that point the repo stops looking like an isolated travel planner and starts looking like a service platform.

Chapter 28. Extension Exercises

One useful exercise is to add a new specialist that enriches event or festival timing for destinations. To do that well, a learner would need to extend schemas, provider access, orchestration, persistence traces, and frontend rendering. This makes the extension a real architectural exercise rather than a prompt tweak.

Another exercise is to deepen the clarification copilot for solo-traveler safety or special-occasion planning. That would force the learner to think about what data should remain free text and what should become structured profile fields that downstream specialists can consume.

A third exercise is to improve operator review. For example, add a richer plain-English explanation layer over governance flags and evidence categories. That exercise teaches how to convert technical runtime metadata into user-trust language.

A fourth exercise is to add evaluation gates that compare new planner outputs against regression expectations. That reinforces the idea that agentic iteration should be measured, not only observed.

Chapter 29. Directory Encyclopedia

The next section explains every project-owned directory in the repository, excluding external, generated, or cache-heavy paths. The goal is not only to tell you what a folder contains, but what role it plays in the architecture and why that role belongs there rather than somewhere else.

Use this chapter as a navigation map while reading the code. When you enter a directory, ask whether it is primarily about boundary handling, business meaning, infrastructure, orchestration, or presentation. That habit will help you preserve the repository's architecture when you extend it.

Chapter 30. File Encyclopedia

The appendix after the directory encyclopedia explains every project-owned file that is currently part of the authored repository surface. This includes documentation, scripts, backend modules, frontend modules, migrations, and tests. Generated and third-party files are excluded on purpose because the aim of this book is to teach the system you own, not the dependencies you downloaded.

File-by-file explanation matters because architecture is not only seen in major diagrams. It is also seen in how small modules are named, where responsibilities live, and what kinds of changes become safe or unsafe as a result. Read this appendix slowly. It is often where maintainability insight becomes concrete.

Chapter 31. End-To-End Request Lifecycle Walkthrough

A complete trip-planning run starts in the browser, but the real lifecycle begins slightly earlier in the user's head. The user forms an intent such as "plan my Darjeeling trip" or "help me produce a honeymoon itinerary with food and photo ideas." The New Trip screen translates that intent into an initial typed request. This is the first important conversion in the system: from messy human desire into machine-usable structure.

The frontend then persists the draft request locally and moves into clarification. The clarification copilot asks a small number of high-signal questions. Each answer is merged back into the normalized request and the clarification profile. At this stage the system is still relatively cheap. It is spending a little reasoning effort to reduce later ambiguity.

Once planning begins, the backend creates workflow runtime records, optionally a queue job, and a planner context. The coordinator or graph runtime then starts specialist work. Research creates context. Itinerary creates the backbone artifact. Parallel specialists enrich that backbone. Budget and review consume the combined picture. Governance decides whether the result is acceptable, low-confidence, or in need of escalation.

The frontend polls run status, displays progress, and eventually fetches the trip artifact set. Screens render research, itinerary, budget, and review from typed backend payloads. In a healthy design, the data path from browser to backend and back feels long but traceable. This repo is valuable because the path is actually inspectable.

Chapter 32. Workflow Runtime Semantics

Workflow runtime is not just execution order. It is the meaning of status transitions, step boundaries, retries, cancellation, reruns, and visible progress. `workflow_runtime_service.py` and the run/step schemas define that meaning for this repository.

A run is the container for one workflow attempt. A step is one observable portion of work within that run. A job is the queue-level execution handle. Those three entities are related but not interchangeable. Many systems fail because they collapse them into one concept.

The repository also surfaces user-facing consequences of runtime semantics. A failed run can be rerun. An active run can be cancelled. A successful run causes navigation into artifact screens. A dead-lettered run remains an important record rather than disappearing from view. These are product outcomes built on backend runtime design.

Students should learn to ask not only "does the workflow run" but "what are the first-class nouns of execution." This repo answers that with trip, job, run, step, audit event, and artifact. That vocabulary is a major part of its engineering maturity.

Chapter 33. Frontend Screen-By-Screen Tour

The Dashboard screen is the orientation layer. It gives the user a landing experience and a route into trip creation or review of existing artifacts. Good dashboards are not only visual; they establish the product's mental model.

The New Trip screen is the intake surface. It should remain structured, readable, and opinionated enough to capture the minimum viable planning contract. Its job is not to do all the planning itself.

The Clarification screen is where conversational specificity enters. It is the bridge between form input and workflow launch. The runtime panel belongs naturally here because this is where the user understands that the system has moved from collection to execution.

Research, Itinerary, Budget, and Review are artifact screens. They exist because different output categories deserve different reading modes. Research is evidence and context. Itinerary is temporal flow. Budget is cost structure. Review is quality and governance. Splitting them improves comprehension and teaches a useful UI design pattern for agentic systems.

Chapter 34. Database Table And Migration Story

A useful way to read the database layer is chronologically. The earliest migrations create trips and jobs because those are the minimum viable durable units. Later migrations add trip state and audit events because the system needs traceability. After that come workflow steps and workflow runs, which make execution observable instead of implicit. Approval and evidence fields arrive when human review and trust become product concerns. Users arrive when identity becomes part of the product.

This order mirrors how many real systems mature. They start by storing what they must. Then they store enough to debug. Then they store enough to explain. Then they store enough to govern. This repository teaches that arc unusually well through its migration history.

The table story also teaches why persistence design should evolve with product needs. If the team had prematurely designed a huge schema before the workflow model stabilized, much of it would have been waste. If it had avoided migrations entirely, it would now lack operational memory. The middle path visible here is the realistic one.

Readers should therefore treat migrations as part of the architecture narrative, not as boring support files. They are the written history of how the software's assumptions changed.

Chapter 35. Test-Driven Reading Guide

One efficient way to understand the repository is to read it test first. Start with `test_multi_agent_coordination.py` to see what the coordinator runtime promises. Then read the runtime itself. Continue with `test_workflow_runtime.py` and the workflow service. Move to `test_observability.py` and the observability service. This pattern turns reading into contract tracing instead of file wandering.

Tests also reveal what the maintainers considered risky. Multi-agent coordination, provider client behavior, API security, operator review, runtime resilience, and destination research are all covered because they are failure-prone surfaces. That risk map is educational.

Another benefit of test-first reading is that it helps learners distinguish essential behavior from implementation detail. A test usually encodes the behavior that must remain true, while the implementation may change shape later. Reading both gives a stronger sense of what can safely be refactored.

In commercial engineering, test suites are part of institutional memory. This repository already uses them that way, and that makes them worth studying as documents rather than only as gatekeepers.

Chapter 36. Recommended Refactoring Principles For This Repo

Refactor by seam, not by ambition. The repository already has clear seams such as route layer, service layer, provider layer, runtime layer, and frontend client layer. Use those seams to localize change. Large cross-cutting rewrites are much riskier in a project that already has multiple execution modes.

Preserve typed contracts while changing internals. If you can improve a specialist prompt, storage implementation, or queue behavior without breaking schemas and response meanings, the system remains teachable and stable. If you change contracts casually, every layer becomes a moving target.

Prefer additive migration over destructive replacement. The coexistence of the sequential graph and coordinator runtime is a healthy example. New behavior was added beside old behavior long enough to validate the new path. That is a stronger pattern than deleting the old system before the new one is operationally trustworthy.

Keep user trust features in mind during refactors. Progress, evidence, governance notes, and review summaries are not cosmetic. Do not simplify backend behavior in a way that makes the system less explainable.

Chapter 37. Guided Study Questions

Use the questions in this chapter as active-learning prompts while reading the codebase. Do not rush through them. The goal is to force architectural reasoning, not to collect trivia.

1. Why does this project need both Postgres and Redis. Which behaviors would break or become far less usable if either were removed.

2. What is the difference between a trip, a job, a run, and a run step. Where is each represented in code and why should they remain distinct.

3. Why is `TripRequest` stricter than a simple free-form prompt. What kinds of planner instability does that strictness reduce.

4. Why does the clarification copilot exist before the main workflow. What quality or cost problems is it trying to avoid.

5. How does the repo keep external provider details from leaking directly into route logic or UI code.

6. Why does the original sequential graph still matter even after the coordinator runtime exists.

7. Which specialist outputs are most reusable across screens, and which are mostly operational.

8. Why is the itinerary artifact the natural backbone for later stay, transport, and food work.

9. What makes the coordinator runtime more autonomous than the original graph, and what constraints keep it from becoming chaotic.

10. Why are delegation rules and topology files worth having explicitly instead of leaving agent choice implicit inside prompts.

11. What does the audit layer add that plain logs do not.

12. Why is observability a product concern here rather than only an SRE concern.

13. Which backend files would you inspect first if the Clarification page worked but the workflow never started.

14. Which files would you inspect if the workflow ran but the Review page stayed empty.

15. Why are application mappers useful even when they sometimes feel repetitive.

16. What kinds of changes belong in the domain layer, and what kinds should stay in services or application schemas.

17. Why do migrations tell an architectural story in this repository.

18. Which files would likely change if you introduced a new provider for hotels or events.

19. Which parts of the stack would need updates if you wanted to expose booking links more safely and consistently.

20. Why is the frontend `planner.ts` file one of the highest-leverage places in the UI codebase.

21. How does route-level UI code differ from reusable component code in terms of acceptable business logic.

22. What evidence in the repository suggests a move toward commercial-grade concerns rather than only demo output.

23. Which current areas appear strongest from a maintainability perspective, and which still look transitional.

24. If you had to add one more human-review checkpoint, where in the workflow would it create the most value.

25. What does this repository teach about the difference between model capability and system capability.

26. Why is progress visibility important in a planner that may take multiple steps and external calls.

27. How would you test that parallel specialist execution improves latency without reducing output quality.

28. Which file is most central for understanding provider configuration, and which is most central for understanding runtime wiring.

29. What role does typed frontend state play in preventing silent UI breakage.

30. If a governance flag is unreadable to users, where should that be translated into plain language: prompt layer, service layer, frontend layer, or all three.

Chapter 38. Guided Code-Reading Prompts

The prompts below are intended for slower, deeper study sessions. Pick one prompt, open the referenced files, and write down your own explanation before checking implementation details.

Prompt 1. Read `docker-compose.yml`, `backend/src/bootstrap.py`, and `backend/src/services/readiness_service.py`. Explain how infrastructure readiness is enforced from container start through backend self-check.

Prompt 2. Read `backend/src/api/main.py` and trace every route to its service or use-case dependency. Which routes are user-facing, which are operator-facing, and which are purely operational.

Prompt 3. Read `backend/src/agents/travel_planner/schemas.py` and list every field in `TripRequest` that protects downstream planning quality.

Prompt 4. Read `backend/src/services/clarification_copilot_service.py` and explain how answers become a `clarification_profile` and how that profile re-enters the normalized request.

Prompt 5. Compare `backend/src/agents/travel_planner/graph.py` with `backend/src/agents/travel_planner/multi_agent/runtime.py`. Which problems does the second solve that the first cannot solve cleanly.

Prompt 6. Read `backend/src/agents/travel_planner/multi_agent/topology.py` and `schemas.py`. Explain why explicit roles and allowed delegations are safer than implicit specialist-to-specialist calls.

Prompt 7. Read `backend/src/agents/travel_planner/multi_agent/adapters.py`. Explain how the repo avoids rewriting all specialist logic while still evolving the runtime model.

Prompt 8. Read `backend/src/services/workflow_runtime_service.py` and map where runs, jobs, and step records are created or updated.

Prompt 9. Read `backend/src/persistence/postgres/trip_repository.py` together with `backend/src/db/models.py`. Explain how planner state is persisted durably.

Prompt 10. Read `backend/src/services/observability_service.py`. What product or operator questions can this service answer.

Prompt 11. Read `backend/src/services/audit_service.py` and related application schemas. What is the minimum useful audit event in this system.

Prompt 12. Read `backend/src/services/auth_service.py` and `backend/src/core/security.py`. Explain how identity and authorization remain separate but connected.

Prompt 13. Read `frontend/src/lib/planner.ts`. Identify which parts are plain fetch wrappers and which parts embody actual client-state policy.

Prompt 14. Read `frontend/src/app/new-trip/NewTrip.tsx` and `frontend/src/app/clarification/Clarification.tsx`. Describe how responsibility is split between generic intake and guided refinement.

Prompt 15. Read `frontend/src/app/research/Research.tsx`, `itinerary/Itinerary.tsx`, `budget/Budget.tsx`, and `review/Review.tsx`. Explain how each page maps to a backend artifact family.

Prompt 16. Read `backend/tests/test_multi_agent_coordination.py`. Which runtime promises are encoded there that a refactor must not break.

Prompt 17. Read `backend/tests/test_reference_links.py`. Why are reference-link and enrichment tests valuable in a travel product.

Prompt 18. Read `backend/tests/test_observability.py` and `test_operator_review.py`. How do these tests reveal product maturity beyond basic planning output.

Prompt 19. Read the Alembic migration history in chronological order. Write a short narrative of how the product matured over time.

Prompt 20. Read the docs set under `docs/` and compare it with implementation. Which docs look like stable foundations and which would need refreshing if the product evolved again.

Chapter 39. Glossary Of Project Terms

This glossary exists so learners can keep the repository's vocabulary stable in their heads while moving across layers. Agentic systems become harder than they need to be when different words are used loosely for the same concept.

Trip. The user-facing planning unit. A trip represents the core planning request and its resulting artifacts.

Job. The queue-facing execution unit. A job is how asynchronous planner work is scheduled and tracked in the background worker infrastructure.

Run. A workflow attempt for a trip. A run has lifecycle state and can succeed, fail, be cancelled, or be rerun.

Run step. One observable segment of a run, such as clarification, research, itinerary, budget, review, or governance.

Clarification profile. Structured user-preference refinement derived from copilot questions and answers after the base trip form.

Planner context. The shared runtime object carrying trip request data plus produced artifacts and status information.

Planner state. The graph-shaped state representation used by LangGraph execution layers.

Specialist. A bounded unit of planner work such as research, itinerary, stay, transport, or food recommendation.

Coordinator. The runtime decision-maker that selects the next specialist or batch of specialists based on the coordination ledger.

Coordination ledger. The typed runtime memory for the custom multi-agent system, including roles, tasks, messages, artifacts, and iteration metadata.

Task board. The list of specialist tasks the coordinator is tracking, including completion and revision states.

Artifact. A structured output such as destination research, itinerary plan, budget assessment, or review assessment.

Evidence. Human-readable support for why a recommendation, issue, or review note exists.

Governance flag. A machine-readable warning or policy signal that indicates low confidence, budget mismatch, or another review-relevant concern.

Audit event. A durable record of meaningful system activity such as request receipt, node completion, or fallback usage.

Observability. The broader runtime visibility layer including traces, metrics, alerts, and run inspection.

Provider. An external API or model service such as OpenAI, Tavily, WeatherAPI, SerpApi, or Aviationstack.

Tooling layer. The planner-owned abstraction layer that turns provider capabilities into governed specialist tools.

Mapper. A module that converts one representation into another, especially from persistence or domain shapes into API response schemas.

Repository. A storage abstraction that persists and retrieves domain objects or records without leaking storage details upward.

Migration. A forward-moving database schema change stored in Alembic history.

Readiness. A stronger form of health checking that verifies dependencies needed for useful service behavior are actually available.

Rerun. A user- or operator-triggered attempt to execute the workflow again after failure, cancellation, or another non-terminal state.

Dead-lettered run. A run that could not be completed successfully even after the expected retry or execution path and is preserved for diagnosis.

Fan-out. The point in a workflow where independent specialists can run in parallel.

Fan-in. The later stage where combined outputs must be reviewed or aggregated before progress can continue.

Product artifact screen. A frontend page such as Research, Itinerary, Budget, or Review that maps directly to a backend artifact family.

Contract drift. The mismatch that appears when one layer changes request, response, or state expectations without updating its consumers.

Commercial readiness. The degree to which the system is hardened enough in quality, identity, governance, observability, and operational behavior for real-world use.

Chapter 40. Final Reading Checklist

Before you consider this repository fully understood, make sure you can answer the following from memory. What starts the workflow. What stores durable planner state. What emits runtime progress. What converts provider output into planner-owned artifacts. What screen shows clarification. What layer owns auth policy. What layer owns database migrations. What layer owns operator review. What file wires the backend together. What file wires the frontend API contract together.

Next, prove your understanding by tracing one feature across every layer. A good example is clarification-driven itinerary quality. Start at the New Trip and Clarification pages, move through the clarification copilot API, inspect the schema changes in `TripRequest`, check where specialist nodes consume the clarified request, inspect persistence and run traces, and finally confirm how the itinerary page renders the result. If you can do that cleanly, you are no longer only reading files; you are reasoning about the system.

Finally, decide what kind of engineer this repository is training you to be. If you only see prompts and output, you are reading it too shallowly. If you can explain the queue, the contracts, the runtime, the evidence, the review surface, the docs, and the test suite as one coherent service, then the project has done its job as a teaching artifact.

Chapter 41. Commercial Hardening Checklist

This final appendix turns the book from a repository explanation into an execution checklist. Use it when deciding what work would move the project from strong prototype toward credible product.

Item 1. Stabilize identity. Replace lightweight auth with stronger password policy, secure session lifecycle, and clearer administrative controls.

Item 2. Add profile ownership rules. Trips, runs, and review decisions should be scoped unambiguously to the acting user or operator role.

Item 3. Expand audit retention rules. Decide how long request, run, and operator data should live and how deletion or redaction should work.

Item 4. Improve source credibility scoring. Not every travel link deserves equal trust, so evidence quality needs ranking, not only capture.

Item 5. Introduce cache strategy for provider calls. Repeated destination research should not always pay the same cost and latency.

Item 6. Add provider-budget controls. Token and API spending limits should be explicit per run, per user, and per environment.

Item 7. Tighten queue idempotency. Reruns, retries, and cancellations should never produce duplicate durable artifacts silently.

Item 8. Strengthen dead-letter handling. Failed runs should be diagnosable and triage-friendly rather than just terminal.

Item 9. Improve frontend empty-state design. Operational errors should become understandable user states rather than raw exceptions.

Item 10. Add richer run analytics. Completion rate, median latency by stage, provider failure rates, and clarification completion rates would improve product insight.

Item 11. Mature approval workflows. Operator review should support note history, rationale capture, and possibly structured approval reasons.

Item 12. Add a source-normalization policy. The same venue, attraction, or recommendation should not appear as multiple low-quality duplicates.

Item 13. Improve itinerary temporal realism. Time budgeting, transit duration, and reservation windows should be validated more rigorously.

Item 14. Extend budget transparency. Users should be able to see per-day and per-category tradeoffs more clearly.

Item 15. Deepen safety personalization. Solo travel, women safety, mobility needs, and after-dark comfort can become stronger structured planning dimensions.

Item 16. Add stronger observability dashboards. The current service layer is a foundation; production needs faster cross-run inspection.

Item 17. Introduce data export policy. Review what trip artifacts can be exported, in what format, and with what privacy guarantees.

Item 18. Add disaster recovery documentation. Backups, schema recovery, and queue recovery should be documented and tested.

Item 19. Separate staging and production secrets more aggressively. Operational boundaries should be explicit and enforced.

Item 20. Increase frontend test coverage. Screen-level tests around clarification, run polling, and tab navigation would catch common regressions earlier.

Item 21. Add end-to-end smoke tests. A single automated happy-path workflow would give strong confidence after infrastructure or contract changes.

Item 22. Improve accessibility review. Keyboard behavior, color contrast, readable labels, and screen-reader semantics should be audited deliberately.

Item 23. Add booking-link validation. External links should be checked for relevance, safety, and deduplication before user exposure.

Item 24. Expand governance messaging. Machine-readable flags must consistently become human-readable explanations in the UI.

Item 25. Add tenant or organization boundaries if the product becomes multi-user at scale. Shared infrastructure is not the same as shared data rights.

Item 26. Create release quality gates specific to agent behavior. Traditional tests alone will not fully protect prompt and output quality.

Item 27. Add artifact-level versioning. It should be possible to compare how itinerary, budget, or review outputs changed across reruns.

Item 28. Build explicit rollback rules for model or provider changes. When a provider degrades, the system should know how to fall back safely.

Item 29. Add stronger documentation ownership. Architecture docs need a maintenance habit so they remain reliable as the code evolves.

Item 30. Decide the business boundary of the product. A commercially coherent planner needs clear answers about who pays, what is guaranteed, and where human review fits into the promise.

Chapter 42. Module-by-Module Teaching Notes

This appendix is designed for teachers, interviewers, and self-learners who want to turn the repository into a guided code-reading exercise. Each paragraph below explains how to use one major module cluster as a teaching surface.

Start with `README.md` as a contract, not a greeting. Ask whether the startup commands, environment story, and architecture claims in the README still match the code. This teaches students that documentation is part of software quality, not peripheral prose.

Use `docker-compose.yml` to teach runtime topology. It is a small file, but it shows service decomposition, dependency order, environment propagation, port mapping, and the relationship between web, worker, queue, and database processes.

Use `backend/src/main.py` and `backend/src/bootstrap.py` to teach composition roots. These files answer a critical design question: where does the application actually become an application rather than a pile of modules.

Use `backend/src/api/main.py` to teach boundary discipline. It demonstrates how user-facing capability, operator-facing capability, and operational capability can coexist in one API surface without collapsing into one undifferentiated controller layer.

Use `backend/src/core/config.py` to teach why settings should be typed. The codebase becomes easier to reason about when configuration is explicit and centrally validated instead of fetched ad hoc.

Use `backend/src/core/security.py` to teach authorization as a contract. A route-level role check is not glamorous, but it is a crisp place to show how business capability and identity constraints intersect.

Use `backend/src/agents/travel_planner/schemas.py` to teach information architecture. Students should ask what fields are first-class, what remains free text, and how that choice shapes every downstream planner result.

Use `backend/src/services/clarification_copilot_service.py` to teach low-cost intelligence before expensive workflow execution. This file is a good antidote to the habit of running a heavy agent graph before the request is actually well-specified.

Use `backend/src/agents/travel_planner/graph.py` to teach state-machine fundamentals. It is linear enough to be understandable, and still realistic enough to show actual workflow branching and node contracts.

Use `backend/src/agents/travel_planner/nodes.py` to teach bounded specialist work. Every specialist here should be read as an artifact-producing service with an LLM and provider substrate, not as a mysterious personality.

Use `backend/src/agents/travel_planner/multi_agent/runtime.py` to teach the transition from workflow to coordination. This is the file that best demonstrates that multi-agent architecture is

really about task control, not merely adding more prompts.

Use `backend/src/agents/travel_planner/multi_agent/coordinator.py` to teach planner authority. The coordinator is not doing all the work; it is deciding who should work, when, and under what conditions.

Use `backend/src/agents/travel_planner/multi_agent/topology.py` to teach policy. Delegation matrices are concrete examples of how autonomy becomes governable.

Use `backend/src/agents/travel_planner/multi_agent/adapters.py` to teach migration strategy. Adapters let teams evolve runtime architecture without discarding working business logic.

Use `backend/src/services/workflow_runtime_service.py` to teach the operational meaning of workflow nouns. This file connects trip intent to job creation, run records, step records, queue submission, and completion handling.

Use the Postgres repositories to teach persistence isolation. Students should compare repository interfaces with SQLAlchemy models and notice how storage mechanics are kept below business-facing services.

Use the migration files to teach architectural history. A codebase often reveals its priorities by what it persisted first and what it added later.

Use `backend/src/services/observability_service.py` and related schemas to teach the difference between logs and operator-facing observability. A metric or run trace only becomes useful when it answers a question someone actually has.

Use `backend/src/services/operator_review_service.py` to teach human-in-the-loop design. This is the module cluster where automation explicitly leaves room for oversight rather than pretending confidence is always enough.

Use `backend/src/providers/factory.py` to teach dependency factories. The logic is simple, but the pattern is strategic because it decouples provider construction from usage sites.

Use `backend/src/agents/travel_planner/tooling` to teach how provider capabilities are transformed into planner-owned tools. This is where policy and maintainability begin to outrank convenience.

Use the backend tests to teach behavior preservation. Encourage students to read a test file before editing the corresponding subsystem. It trains them to look for promises instead of just implementation details.

Use `frontend/src/lib/planner.ts` to teach frontend integration architecture. It is one of the few files where transport logic, local persistence, and typed UI contracts all meet.

Use `frontend/src/app/new-trip/NewTrip.tsx` and `frontend/src/app/clarification/Clarification.tsx` together to teach staged user journeys. The contrast between base intake and refinement is a stronger lesson than either screen alone.

Use `frontend/src/app/research`, `itinerary`, `budget`, and `review` to teach artifact-oriented UI design. Each page reads a different backend artifact family through a different user-comprehension lens.

Use the reusable component folder to teach presentational decomposition. Students should notice which pieces are truly generic and which are becoming domain-heavy enough to deserve relocation.

Use `docs/architecture.md`, `docs/layered-architecture.md`, and `docs/true-multi-agent-architecture.md` to teach architectural self-awareness. This repository does not hide its current-versus-target state, and that is a professional habit worth copying.

Use `.github/workflows` to teach that a repository is also a delivery system. Code quality, deployability, and secret scanning are part of what the team is building, not separate from it.

Finally, use this whole repository to teach a meta-lesson: the strongest agentic systems are not the ones with the longest prompts. They are the ones where prompts, data, orchestration, storage, UI, observability, and governance all fit together without confusion.

Chapter 43. Design Principles To Carry Forward

Principle 1. Keep the request model honest. If a future feature needs structured input, add the structure instead of shoving more meaning into a free-text notes field.

Principle 2. Preserve the separation between orchestration and specialist execution. The coordinator should decide who acts; specialists should focus on producing artifacts.

Principle 3. Prefer explicit runtime nouns. Jobs, runs, steps, audit events, and artifacts are worth naming because they simplify debugging and explanation.

Principle 4. Treat frontend state as part of the architecture. Local persistence, active run ids, and clarification state are not random UI details; they shape the user's mental model of the system.

Principle 5. Convert machine metadata into human-readable product language. Governance flags are operationally useful only when users and reviewers can understand their meaning quickly.

Principle 6. Reuse working logic through adapters when architecture evolves. Full rewrites are tempting, but careful adaptation is often how systems mature without losing reliability.

Principle 7. Keep provider instability behind owned contracts. External APIs should feel replaceable from the perspective of the planner and the UI.

Principle 8. Use tests to describe promises, not only to catch syntax mistakes. The most useful tests in this repo define what the runtime is allowed to do.

Principle 9. Build docs that explain both the present and the target. Architectural evolution becomes safer when teams admit what is transitional.

Principle 10. Make review and observability first-class. Agentic software becomes credible when humans can inspect how decisions were produced and where confidence is weak.

Principle 11. Optimize latency where dependencies allow, not everywhere blindly. The selective parallelism in this repo is a better model than naive all-at-once concurrency.

Principle 12. Keep the repository teachable. Clear naming, layered boundaries, and documented intent are strategic assets, especially in systems that combine deterministic code with probabilistic model behavior.

Directory Encyclopedia

The entries below explain every project-owned directory that meaningfully participates in the authored repository surface.

`.github`

Automation folder for GitHub-integrated delivery concerns such as CI, deployment, and secret scanning. This is where repository governance leaves the application runtime and becomes delivery workflow.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`.github/workflows`

Pipeline definitions executed by GitHub Actions. These files encode what the project considers a healthy push, a deployable build, and a secure release motion.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend`

Python application boundary. Everything here supports the FastAPI API, the planner runtime, the coordinator-led multi-agent backend, the persistence layer, and backend-specific testing.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/alembic`

Database migration framework directory. Alembic tracks schema evolution so environments can move forward in a controlled and replayable way.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/alembic/versions`

Concrete database migration history. Each file explains one structural change to tables, columns, runtime tracking, or approval metadata.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/evals`

Evaluation artifact folder. This holds static regression data used to measure whether planner behavior drifts over time.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/scripts`

Utility scripts scoped to backend operations. These are not request-path code; they support testing, load checks, or manual operations.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src`

Backend source root. The code beneath this directory follows a layered architecture: API, application, domain, persistence, providers, services, messaging, workers, and agent runtime.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/agents`

Planner intelligence layer. This directory contains the original sequential workflow graph, the node implementations, the multi-agent coordination runtime, and the tool abstractions used by specialists.

When working in this area, trace how changes affect schemas, runtime progress, confidence signals, and downstream UI rendering. Planner logic is rarely isolated; it usually impacts observability and user trust.

`backend/src/agents/travel_planner`

Planner intelligence layer. This directory contains the original sequential workflow graph, the node implementations, the multi-agent coordination runtime, and the tool abstractions used by specialists.

When working in this area, trace how changes affect schemas, runtime progress, confidence signals, and downstream UI rendering. Planner logic is rarely isolated; it usually impacts observability and user trust.

`backend/src/agents/travel_planner/multi_agent`

Planner intelligence layer. This directory contains the original sequential workflow graph, the node implementations, the multi-agent coordination runtime, and the tool abstractions used by specialists.

When working in this area, trace how changes affect schemas, runtime progress, confidence signals, and downstream UI rendering. Planner logic is rarely isolated; it usually impacts observability and user trust.

`backend/src/agents/travel_planner/tooling`

Planner intelligence layer. This directory contains the original sequential workflow graph, the node implementations, the multi-agent coordination runtime, and the tool abstractions used by specialists.

When working in this area, trace how changes affect schemas, runtime progress, confidence signals, and downstream UI rendering. Planner logic is rarely isolated; it usually impacts observability and user trust.

`backend/src/api`

HTTP boundary for the backend. Files here translate requests into application/service calls and turn internal models into response contracts.

Changes here should be reviewed against auth policy, response models, and frontend consumers. Route files are small, but contract drift here breaks the product quickly.

`backend/src/api/dependencies`

HTTP boundary for the backend. Files here translate requests into application/service calls and turn internal models into response contracts.

Changes here should be reviewed against auth policy, response models, and frontend consumers. Route files are small, but contract drift here breaks the product quickly.

`backend/src/api/routes`

HTTP boundary for the backend. Files here translate requests into application/service calls and turn internal models into response contracts.

Changes here should be reviewed against auth policy, response models, and frontend consumers. Route files are small, but contract drift here breaks the product quickly.

`backend/src/application`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/admin`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/audit`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/auth`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/jobs`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/observability`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/trips`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/application/workflows`

Application contracts and use cases. This layer maps domain and persistence models into API-safe schemas without embedding transport logic into the domain.

This layer is the right place to enforce stable transport contracts. If you add a new backend capability, check whether it needs a schema, mapper, or use-case update here.

`backend/src/core`

Cross-cutting backend utilities such as settings, logging, request context, response shaping, and security. These modules are used by many layers but should stay narrowly infrastructural.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/db`

SQLAlchemy database setup. This directory wires the engine, sessions, base model metadata, and relational model declarations.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/domain`

Business domain layer. This is the low-level contract surface for trips, workflows, jobs, and audit concepts, expressed independently from FastAPI and storage details.

Edits in the domain layer should be conservative. If a change is domain-worthy, many outer layers may need to adapt because domain meaning is upstream of storage and transport.

backend/src/domain/audit

Business domain layer. This is the low-level contract surface for trips, workflows, jobs, and audit concepts, expressed independently from FastAPI and storage details.

Edits in the domain layer should be conservative. If a change is domain-worthy, many outer layers may need to adapt because domain meaning is upstream of storage and transport.

backend/src/domain/jobs

Business domain layer. This is the low-level contract surface for trips, workflows, jobs, and audit concepts, expressed independently from FastAPI and storage details.

Edits in the domain layer should be conservative. If a change is domain-worthy, many outer layers may need to adapt because domain meaning is upstream of storage and transport.

backend/src/domain/trips

Business domain layer. This is the low-level contract surface for trips, workflows, jobs, and audit concepts, expressed independently from FastAPI and storage details.

Edits in the domain layer should be conservative. If a change is domain-worthy, many outer layers may need to adapt because domain meaning is upstream of storage and transport.

backend/src/domain/workflows

Business domain layer. This is the low-level contract surface for trips, workflows, jobs, and audit concepts, expressed independently from FastAPI and storage details.

Edits in the domain layer should be conservative. If a change is domain-worthy, many outer layers may need to adapt because domain meaning is upstream of storage and transport.

backend/src/evals

Runtime evaluation logic. These modules know how to score outputs and compare behavior against golden examples.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

backend/src/messaging

Async execution infrastructure. Queue wiring and background-job helpers live here so the API does not need to know queue mechanics.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

backend/src/models

Project directory `backend/src/models`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/persistence`

Repository implementations. Memory repositories enable lightweight behavior and tests; Postgres repositories provide durable storage.

Repository changes should be paired with storage-oriented tests and migration awareness. Persistence bugs often surface as missing traces or stale UI state much later in the flow.

`backend/src/persistence/memory`

Repository implementations. Memory repositories enable lightweight behavior and tests; Postgres repositories provide durable storage.

Repository changes should be paired with storage-oriented tests and migration awareness. Persistence bugs often surface as missing traces or stale UI state much later in the flow.

`backend/src/persistence/postgres`

Repository implementations. Memory repositories enable lightweight behavior and tests; Postgres repositories provide durable storage.

Repository changes should be paired with storage-oriented tests and migration awareness. Persistence bugs often surface as missing traces or stale UI state much later in the flow.

`backend/src/persistence/sqlite`

Repository implementations. Memory repositories enable lightweight behavior and tests; Postgres repositories provide durable storage.

Repository changes should be paired with storage-oriented tests and migration awareness. Persistence bugs often surface as missing traces or stale UI state much later in the flow.

`backend/src/providers`

External provider client layer. These modules define how the application talks to OpenAI, Tavily, SerpApi, Aviationstack, and WeatherAPI.

This area should absorb third-party API variability so the rest of the application can stay stable. Prefer normalization and explicit error behavior over leaking raw provider payloads upward.

`backend/src/repositories`

Project directory `backend/src/repositories`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/services`

Service layer for orchestrating higher-level backend behaviors. These modules usually combine repositories, runtimes, and application policy into use-ready operations.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/utils`

Project directory `backend/src/utils`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/src/workers`

Long-running background worker processes. Code here is executed by the queue worker rather than the API web server.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`backend/tests`

Backend test suite. These files are the proof surface for architecture contracts, coordinator behavior, provider wiring, observability, and runtime resilience.

Use this directory not only for regression prevention but as executable documentation. A strong habit is to read the relevant test before modifying a subsystem.

`docs`

Project documentation folder. It captures architecture, agent flow, API contract, operational runbooks, and deployment notes that explain the current and target system.

Documentation changes should track the real system state. If implementation evolves, stale docs become architectural debt rather than support material.

`docs/assets`

Static documentation assets such as the layered architecture image used by the docs and by this handbook.

Documentation changes should track the real system state. If implementation evolves, stale docs become architectural debt rather than support material.

`docs/operations`

Operations guide collection. These pages focus on SLOs, deployment, resilience, and runbook-style production behavior.

Documentation changes should track the real system state. If implementation evolves, stale docs become architectural debt rather than support material.

`frontend`

React application boundary. This folder contains the web UI, frontend build tooling, asset pipeline, and client-side state logic.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/public`

Public frontend shell assets served as-is by the React app. This is where the HTML entry document sits.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src`

Frontend source root. The code here is split into app screens, reusable components, layouts, assets, and client libraries.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/app`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/budget`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/clarification`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/dashboard`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/itinerary`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/login`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/new-trip`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/research`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/app/review`

Route-level screens. Each file or subfolder under app represents one major user-facing page or a page-specific style sheet.

Route-level changes should be checked against shared components and backend contract assumptions. UI state and workflow-state handling are tightly coupled here.

`frontend/src/assets`

Frontend media and branding assets. These files shape the visual identity of the UI.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/components`

Reusable UI components. These are the frontend building blocks shared across multiple route-level screens.

Component changes can ripple across multiple pages. Keep these pieces reusable and avoid burying route-specific business logic in shared UI blocks.

`frontend/src/features`

Project directory `frontend/src/features`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/hooks`

Project directory `frontend/src/hooks`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/layouts`

Shared page chrome and structural layout components. These files keep route pages visually consistent.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/lib`

Frontend integration library layer. This folder centralizes API calls, localStorage behavior, and shared client-side types.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/services`

Project directory `frontend/src/services`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`frontend/src/types`

Project directory `frontend/src/types`. It groups related implementation artifacts and keeps concerns separated so the repository can scale without becoming a flat, untraceable file list.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

`scripts`

Top-level helper scripts. These tend to coordinate the whole stack rather than only backend or frontend concerns.

Treat this directory as a bounded context. Before editing inside it, identify which adjacent folders consume its outputs so you do not create hidden contract drift.

File Encyclopedia

Each file entry includes its path and a short explanation of why it exists in the system. Read this appendix together with the codebase for the highest learning value.

Repository Root

`.env`

Root environment file for local developer convenience. In this repo it supplements service-level configuration and keeps machine-specific secrets out of source logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`.env.example`

Environment template that shows the shape of required variables without embedding real secrets. It is the safest starting point for new contributors.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`.gitignore`

Git hygiene file describing which local artifacts, caches, secrets, and build outputs should never be committed.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`.nvmrc`

Node version marker that stabilizes frontend tooling across machines.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`.pre-commit-config.yaml`

Pre-commit automation config. This is a defensive quality gate that catches formatting and lint problems before code reaches CI.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`.python-version`

Python version marker so backend tooling runs with the interpreter expected by the repo.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`README.md`

Primary entry document for the whole repository. It introduces the stack, startup flow, runtime dependencies, and documentation map.

Markdown files in this repo are knowledge assets. Keep them aligned with the real code so readers do not have to choose between docs and implementation.

`docker-compose.yml`

Local multi-container stack definition. It expresses how Postgres, Redis, backend, worker, and frontend fit together for development and testing.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`package-lock.json`

Root NPM lock file. It stabilizes JavaScript dependency resolution for any root-level package usage.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

.github/workflows

`.github/workflows/ci.yml`

GitHub Actions workflow for ci. It encodes one delivery or governance automation path that runs outside the application runtime itself.

Workflow changes affect team velocity and deployment risk. Validate the pipeline logic carefully because mistakes here are operational, not only local.

`.github/workflows/deploy-production.yml`

GitHub Actions workflow for deploy production. It encodes one delivery or governance automation path that runs outside the application runtime itself.

Workflow changes affect team velocity and deployment risk. Validate the pipeline logic carefully because mistakes here are operational, not only local.

`.github/workflows/deploy-staging.yml`

GitHub Actions workflow for deploy staging. It encodes one delivery or governance automation path that runs outside the application runtime itself.

Workflow changes affect team velocity and deployment risk. Validate the pipeline logic carefully because mistakes here are operational, not only local.

`.github/workflows/secrets-scan.yml`

GitHub Actions workflow for secrets scan. It encodes one delivery or governance automation path that runs outside the application runtime itself.

Workflow changes affect team velocity and deployment risk. Validate the pipeline logic carefully because mistakes here are operational, not only local.

backend

`backend/.dockerignore`

Repository-owned file `backend/.dockerignore`. It contributes to the documented stack and is included so readers can trace the project end to end instead of only learning the headline files.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/.env`

Backend environment sample or local environment file. These files define how backend configuration changes by environment without changing code.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/.env.dev`

Backend environment sample or local environment file. These files define how backend configuration changes by environment without changing code.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/.env.dev.example`

Backend environment sample or local environment file. These files define how backend configuration changes by environment without changing code.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/.env.prod.example`

Backend environment sample or local environment file. These files define how backend configuration changes by environment without changing code.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/.env.staging.example`

Backend environment sample or local environment file. These files define how backend configuration changes by environment without changing code.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/Dockerfile`

Backend container build recipe. It packages Python dependencies and startup behavior into a repeatable runtime image.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/alembic.ini`

Alembic configuration file that points migration tooling at the backend's migration environment.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/pyproject.toml`

Backend project metadata and tool configuration, including linting and formatting expectations.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/requirements-dev.txt`

Backend Python dependency manifest for development and test packages.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/requirements.txt`

Backend Python dependency manifest for runtime packages.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/alembic

`backend/alembic/env.py`

Alembic runtime environment file. It tells migrations how to connect settings, metadata, and execution context.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/alembic/script.py.mako`

Alembic migration template used when generating new revision files.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/alembic/versions

`backend/alembic/versions/20260509_000001_create_trip_and_job_tables.py`

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/alembic/versions/20260509_000002_add_trip_state_and_audit_events.py

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/alembic/versions/20260509_000003_add_workflow_run_steps.py

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/alembic/versions/20260510_000004_add_trip_approval_and_evidence.py

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/alembic/versions/20260602_000005_add_workflow_runs_and_job_runtime_fields.py

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/alembic/versions/20260611_000006_add_users_table.py

Database schema migration step. This file applies one forward change so the database can evolve in sync with the application.

Never edit a migration casually after it has become part of history. Prefer a new forward migration that explains the next schema step clearly.

backend/evals

backend/evals/regression_baseline.json

Static evaluation baseline that captures expected planner behavior for regression scoring.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/scripts

backend/scripts/load_test.py

Load-testing script used to probe backend endpoints under concurrent request pressure.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src

backend/src/__init__.py

Repository-owned file `backend/src/__init__.py`. It contributes to the documented stack and is included so readers can trace the project end to end instead of only learning the headline files.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/bootstrap.py

Backend composition root. This is where repositories, services, runtimes, provider clients, and dependency injection are wired together.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/main.py

Backend process entrypoint. It creates the FastAPI app, installs middleware, and mounts the API router.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/agents/travel_planner

backend/src/agents/travel_planner/contracts.py

Planner contract declarations used to standardize agent input and output handling.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/governance.py

Planner governance logic that evaluates risk, confidence, and routing conditions.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/graph.py

Legacy sequential LangGraph workflow definition. It shows the original node-by-node planner path and is still important for understanding the repo's evolution.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/nodes.py

Specialist implementation file containing the heavy lifting for planner outputs such as research, itinerary, stay, transport, food, budget, safety, review, and governance.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/prompts.py`

Prompt assets for specialist planning behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/research_clients.py`

Thin research-helper module that coordinates provider access for research tasks.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/research_prompts.py`

Prompt assets focused specifically on research and evidence gathering.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/routing.py`

Routing helper that decides what happens after clarification or other runtime checkpoints.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/schemas.py`

Core planner data contracts. These Pydantic models define request shape, intermediate outputs, and typed response artifacts across the backend and frontend.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/state.py`

Shared planner state model used by graph nodes and by runtime translation logic.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

`backend/src/agents/travel_planner/tools.py`

Higher-level tool entry module for the planner subsystem.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/multi_agent

backend/src/agents/travel_planner/multi_agent/__init__.py

Multi-agent coordination support module.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/multi_agent/adapters.py

Adapter layer that lets the new coordinator runtime reuse the older specialist node implementations without rewriting every agent from scratch.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/multi_agent/coordinator.py

Coordinator agent logic. This file decides what specialist should act next and whether work can proceed in parallel.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/multi_agent/runtime.py

Coordinator-driven multi-agent runtime. It is the most important file for autonomous specialist dispatch, parallel fan-out, result merging, and runtime progress emission.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/multi_agent/schemas.py

Typed coordination ledger contracts: roles, tasks, messages, artifacts, and iteration metadata used by the custom multi-agent runtime.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/multi_agent/topology.py

Role topology and delegation policy. It defines what the agent network is allowed to do and how initial task boards are seeded.

Changes here can alter autonomy, latency, and trace semantics at the same time. Rerun coordinator and runtime tests whenever this area moves.

backend/src/agents/travel_planner/tooling

backend/src/agents/travel_planner/tooling/__init__.py

Planner tool layer module for init . It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/airports.py

Planner tool layer module for airports. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/base.py

Planner tool layer module for base. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/contracts.py

Planner tool layer module for contracts. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/factory.py

Planner tool layer module for factory. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/llm_tools.py

Planner tool layer module for llm tools. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/provider_tools.py

Planner tool layer module for provider tools. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/registry.py

Planner tool layer module for registry. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/search_tools.py

Planner tool layer module for search tools. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/agents/travel_planner/tooling/travel_tools.py

Planner tool layer module for travel tools. It supports tool registration, provider binding, validation, or travel-specific helper behavior.

Edits here often affect planner output quality, evidence shape, and multiple frontend screens. Verify both typed artifacts and user-visible summaries after changes.

backend/src/api

backend/src/api/main.py

FastAPI route module. It exposes backend capability through explicit HTTP endpoints, input validation, and response models.

Any change here should be reviewed against frontend callers, auth requirements, and audit implications. API surfaces are where many hidden backend assumptions become public.

backend/src/application/admin

backend/src/application/admin/schemas.py

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/audit

backend/src/application/audit/mappers.py

Application-layer mapper file. It transforms domain or persistence models into response schemas and vice versa.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/audit/schemas.py

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/auth

backend/src/application/auth/schemas.py

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/jobs

backend/src/application/jobs/mappers.py

Application-layer mapper file. It transforms domain or persistence models into response schemas and vice versa.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/jobs/schemas.py

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/observability

backend/src/application/observability/schemas.py

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/trips

backend/src/application/trips/async_use_cases.py

Application use-case module. It packages a coherent business operation behind a clear execution method.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/trips/mappers.py

Application-layer mapper file. It transforms domain or persistence models into response schemas and vice versa.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/trips/use_cases.py

Application use-case module. It packages a coherent business operation behind a clear execution method.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/application/workflows

`backend/src/application/workflows/mappers.py`

Application-layer mapper file. It transforms domain or persistence models into response schemas and vice versa.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/application/workflows/schemas.py`

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/application/workflows/step_mappers.py`

Application-layer mapper file. It transforms domain or persistence models into response schemas and vice versa.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/application/workflows/step_schemas.py`

Application-layer schema file. It defines transport-safe typed objects passed between use cases and API responses.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/core

`backend/src/core/config.py`

Core backend utility module for config. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/core/logging.py`

Core backend utility module for logging. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/core/redaction.py`

Core backend utility module for redaction. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/core/request_context.py`

Core backend utility module for request context. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/core/response_shaping.py`

Core backend utility module for response shaping. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/core/security.py`

Core backend utility module for security. It supports the rest of the stack with shared infrastructure rather than domain-specific planning logic.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/db

`backend/src/db/base.py`

Database infrastructure module for base. It helps SQLAlchemy talk to the configured Postgres database consistently.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/db/bootstrap.py`

Database infrastructure module for bootstrap. It helps SQLAlchemy talk to the configured Postgres database consistently.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/db/models.py`

Database infrastructure module for models. It helps SQLAlchemy talk to the configured Postgres database consistently.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/db/session.py`

Database infrastructure module for session. It helps SQLAlchemy talk to the configured Postgres database consistently.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/audit

`backend/src/domain/audit/models.py`

Domain layer module for models. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/domain/audit/repositories.py`

Domain layer module for repositories. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/jobs

`backend/src/domain/jobs/models.py`

Domain layer module for models. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/domain/jobs/repositories.py`

Domain layer module for repositories. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/trips

`backend/src/domain/trips/guardrails.py`

Domain layer module for guardrails. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/trips/models.py

Domain layer module for models. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/trips/policies.py

Domain layer module for policies. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/trips/repositories.py

Domain layer module for repositories. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/trips/services.py

Domain layer module for services. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/workflows

backend/src/domain/workflows/models.py

Domain layer module for models. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/domain/workflows/repositories.py

Domain layer module for repositories. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/domain/workflows/step_models.py`

Domain layer module for step models. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/domain/workflows/step_repositories.py`

Domain layer module for step repositories. It describes the stable business concepts the rest of the backend is built around.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/evals

`backend/src/evals/__init__.py`

Evaluation runtime module for init . It supports scoring, regression comparison, or golden-case execution.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/evals/golden_cases.py`

Evaluation runtime module for golden cases. It supports scoring, regression comparison, or golden-case execution.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/evals/regressions.py`

Evaluation runtime module for regressions. It supports scoring, regression comparison, or golden-case execution.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/evals/runner.py`

Evaluation runtime module for runner. It supports scoring, regression comparison, or golden-case execution.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/evals/scoring.py`

Evaluation runtime module for scoring. It supports scoring, regression comparison, or golden-case execution.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/messaging

`backend/src/messaging/background_jobs.py`

Queue and async-messaging module for background jobs. It keeps background execution mechanics separate from request handling.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`backend/src/messaging/redis_queue.py`

Queue and async-messaging module for redis queue. It keeps background execution mechanics separate from request handling.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/src/persistence/memory

`backend/src/persistence/memory/job_repository.py`

In-memory repository implementation. It provides a lightweight non-durable storage adapter useful for tests or simplified execution paths.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

`backend/src/persistence/memory/trip_repository.py`

In-memory repository implementation. It provides a lightweight non-durable storage adapter useful for tests or simplified execution paths.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

`backend/src/persistence/memory/workflow_run_repository.py`

In-memory repository implementation. It provides a lightweight non-durable storage adapter useful for tests or simplified execution paths.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

`backend/src/persistence/memory/workflow_run_step_repository.py`

In-memory repository implementation. It provides a lightweight non-durable storage adapter useful for tests or simplified execution paths.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres

backend/src/persistence/postgres/__init__.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/audit_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/job_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/trip_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/user_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/workflow_run_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/persistence/postgres/workflow_run_step_repository.py

Postgres-backed repository implementation. It persists domain and runtime data durably in the relational database.

Repository changes should be checked against migrations, mappers, and any admin or observability screens that depend on the stored data shape.

backend/src/providers

backend/src/providers/__init__.py

External provider client module for init . It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/providers/base.py

External provider client module for base. It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/providers/factory.py

External provider client module for factory. It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/providers/llm.py

External provider client module for llm. It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/providers/search.py

External provider client module for search. It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/providers/travel.py

External provider client module for travel. It wraps a third-party API or provider-facing abstraction behind repo-owned contracts.

Provider-facing files should hide instability from the rest of the codebase. If you touch them, check timeout behavior, fallback paths, and source normalization.

backend/src/services

backend/src/services/audit_service.py

Backend service module for audit service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/auth_service.py

Backend service module for auth service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/clarification_copilot_service.py

Backend service module for clarification copilot service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/observability_service.py

Backend service module for observability service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/operator_review_service.py

Backend service module for operator review service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/readiness_service.py

Backend service module for readiness service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/travel_planner_service.py

Backend service module for travel planner service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/services/workflow_runtime_service.py

Backend service module for workflow runtime service. It coordinates repositories, runtimes, or policies into a higher-level operation.

Service modules coordinate several dependencies, so interface drift can have wide impact. Review their callers and tests together rather than in isolation.

backend/src/workers

`backend/src/workers/workflow_worker.py`

Worker-process entry module that executes queued planner jobs outside the request-response cycle.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

backend/tests

`backend/tests/conftest.py`

Backend automated test module for conftest. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_api_security.py`

Backend automated test module for test api security. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_application_layers.py`

Backend automated test module for test application layers. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_config.py`

Backend automated test module for test config. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_destination_research_agent.py`

Backend automated test module for test destination research agent. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_domain_layers.py`

Backend automated test module for test domain layers. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_eval_suite.py`

Backend automated test module for test eval suite. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_multi_agent_coordination.py`

Backend automated test module for test multi agent coordination. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_observability.py`

Backend automated test module for test observability. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_operator_review.py`

Backend automated test module for test operator review. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_provider_clients.py`

Backend automated test module for test provider clients. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_provider_live_smoke.py`

Backend automated test module for test provider live smoke. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_reference_links.py`

Backend automated test module for test reference links. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_runtime_resilience.py`

Backend automated test module for test runtime resilience. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_tooling_layer.py`

Backend automated test module for test tooling layer. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

`backend/tests/test_workflow_runtime.py`

Backend automated test module for test workflow runtime. It verifies that a particular backend concern remains correct as the code evolves.

If this test fails after a change, treat that as a contract warning rather than an inconvenience. The test suite in this repo is one of the clearest descriptions of intended backend behavior.

docs

`docs/agent-communication.md`

Project documentation chapter focused on agent communication. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/agent-flow.md`

Project documentation chapter focused on agent flow. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/api-contract.md`

Project documentation chapter focused on api contract. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/architecture.md`

Project documentation chapter focused on architecture. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/layered-architecture.md`

Project documentation chapter focused on layered architecture. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/true-multi-agent-architecture.md`

Project documentation chapter focused on true multi agent architecture. It exists so architectural knowledge is not trapped only inside code.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

docs/assets

`docs/assets/layered-architecture.png`

Documentation asset used to visually explain the system. This is not executable code, but it is important for architectural communication.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

docs/operations

`docs/operations/deployment.md`

Operations handbook page about deployment. It teaches how the system should be deployed, monitored, or recovered when running as a service.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/operations/load-and-resilience.md`

Operations handbook page about load and resilience. It teaches how the system should be deployed, monitored, or recovered when running as a service.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/operations/runbooks.md`

Operations handbook page about runbooks. It teaches how the system should be deployed, monitored, or recovered when running as a service.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

`docs/operations/slo.md`

Operations handbook page about slo. It teaches how the system should be deployed, monitored, or recovered when running as a service.

Use docs files to preserve design intent. If the implementation changes, updating this file is part of finishing the work rather than optional cleanup.

frontend

`frontend/.dockerignore`

Frontend infrastructure file for frontend Docker build ignore rules.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/.env.local`

Frontend infrastructure file for local frontend environment overrides.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/Dockerfile`

Frontend container build recipe used to package and run the React client in Docker.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/package-lock.json`

Frontend infrastructure file for frontend dependency lock file.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/package.json`

Frontend infrastructure file for frontend package manifest and script registry.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/tsconfig.json`

Frontend infrastructure file for TypeScript compiler configuration.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

frontend/public

`frontend/public/index.html`

Public-facing frontend shell file served directly by the React app bootstrap process.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

frontend/src

`frontend/src/App.tsx`

Frontend application router and top-level page composition entry.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/src/index.css`

Global stylesheet for baseline typography, variables, and cross-app visuals.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/src/index.tsx`

React bootstrap entry that mounts the app into the browser DOM.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/src/react-app-env.d.ts`

TypeScript environment declarations for the React toolchain.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

frontend/src/app

`frontend/src/app/AppPage.module.css`

Page-specific style sheet that gives one route its visual identity and layout behavior.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/budget

`frontend/src/app/budget/Budget.tsx`

Route-level React page for `Budget`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/clarification

`frontend/src/app/clarification/Clarification.tsx`

Route-level React page for `Clarification`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/dashboard

`frontend/src/app/dashboard/Dashboard.module.css`

Page-specific style sheet that gives one route its visual identity and layout behavior.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

`frontend/src/app/dashboard/Dashboard.tsx`

Route-level React page for `Dashboard`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/itinerary

`frontend/src/app/itinerary/Itinerary.tsx`

Route-level React page for `Itinerary`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/login

`frontend/src/app/login/Login.module.css`

Page-specific style sheet that gives one route its visual identity and layout behavior.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

`frontend/src/app/login/Login.tsx`

Route-level React page for `Login`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

`frontend/src/app/login/Signup.tsx`

Route-level React page for `Signup`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/new-trip

`frontend/src/app/new-trip/NewTrip.tsx`

Route-level React page for `NewTrip`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/research

`frontend/src/app/research/Research.tsx`

Route-level React page for `Research`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/app/review

`frontend/src/app/review/Review.tsx`

Route-level React page for `Review`. It is where one major user workflow is composed from shared components and client-side API state.

When editing this screen, confirm the user journey still makes sense end to end. Route files are where backend capability becomes product behavior.

frontend/src/assets

`frontend/src/assets/atlas-brand-mark.svg`

Frontend visual asset used for branding or interface polish.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

frontend/src/components

`frontend/src/components/AccountBar.module.css`

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/AccountBar.tsx

Reusable React component `AccountBar`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/AccountCard.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/AccountCard.tsx

Reusable React component `AccountCard`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Card.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Card.tsx

Reusable React component `Card`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Chat.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Chat.tsx

Reusable React component `Chat`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Checklist.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Checklist.tsx

Reusable React component `Checklist`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/ExportPanel.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/ExportPanel.tsx

Reusable React component `ExportPanel`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/FilterBar.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/FilterBar.tsx

Reusable React component `FilterBar`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Header.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Header.tsx

Reusable React component `Header`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/ItineraryDayCard.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/ItineraryDayCard.tsx

Reusable React component `ItineraryDayCard`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Sidebar.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/Sidebar.tsx

Reusable React component `Sidebar`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/StatusBadge.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/StatusBadge.tsx

Reusable React component `StatusBadge`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/TravelRibbon.module.css

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/components/TravelRibbon.tsx

Reusable React component `TravelRibbon`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

`frontend/src/components/UtilityPanel.module.css`

Component-scoped CSS module. It keeps visual rules local to a reusable UI building block.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

`frontend/src/components/UtilityPanel.tsx`

Reusable React component `UtilityPanel`. This file is a UI building block consumed by one or more route-level pages.

Because this is shared UI, visual or prop-interface changes should be checked across every page that imports the component.

frontend/src/layouts

`frontend/src/layouts/AppLayout.module.css`

Shared layout file that defines app-level chrome, navigation framing, or layout-specific styling.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`frontend/src/layouts/AppLayout.tsx`

Shared layout file that defines app-level chrome, navigation framing, or layout-specific styling.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

frontend/src/lib

`frontend/src/lib/planner.test.ts`

Frontend client test file that verifies planner-library behavior and protects against regressions in UI-facing integration logic.

This client layer is the frontend contract hub. Small mistakes here spread quickly because many pages consume the same helpers and types.

`frontend/src/lib/planner.ts`

Frontend planner client library. It centralizes API calls, request/response types, localStorage behavior, and frontend integration with workflow runtime state.

This client layer is the frontend contract hub. Small mistakes here spread quickly because many pages consume the same helpers and types.

scripts

`scripts/generate_teaching_book_pdf.py`

Repository-owned file `scripts/generate_teaching_book_pdf.py`. It contributes to the documented stack and is included so readers can trace the project end to end instead of only learning the headline files.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

`scripts/start-local-stack.sh`

Developer convenience launcher that brings up the stack, waits for readiness, and opens the browser for quicker manual testing.

The safest way to modify this file is to first identify its nearest consumers and adjacent tests. Small local edits in layered systems often have cross-layer consequences.

Closing Notes

This book is intentionally exhaustive because the repository itself is richer than a small tutorial project. The goal was not only to summarize the architecture but to make the codebase teachable as a system. By the time a reader finishes this handbook, they should understand where runtime logic lives, how data moves, why Redis and Postgres are present, why the coordinator runtime matters, how the UI maps to backend artifacts, and how to locate any major file quickly.

The most important takeaway is not a specific library choice. It is the repository habit of making boundaries visible. Types are visible. Roles are visible. Runtime steps are visible. Review and evidence surfaces are visible. That habit is what lets agentic systems scale from experiments into maintainable software.

Regenerate this PDF whenever the repository evolves. A teaching book should move with the code, not drift away from it.

Reader Notes Page 1

Use this page to write your own map of the repository: the first files you would open to explain request intake, workflow execution, persistence, and frontend rendering to another engineer.

Reader Notes Page 2

Use this page to outline the first production-grade improvements you would ship next. A strong answer should touch product trust, runtime resilience, identity, observability, and evaluation.